

ML LEAP Final Project: Final Project: CNN MNIST

Kyriazis Charitopoulos

October 10, 2026

Abstract

The subject of the present, final assignment of the ML LEAP program, is the development, training, and comparative evaluation of different Convolutional Neural Network (CNN) architectures, aiming at the automatic classification of handwritten digit images. For this purpose, the MNIST dataset is used, which constitutes a benchmark in the field of computer vision. Through this assignment, the effect of increasing the network's depth—specifically with the addition of an extra convolutional layer—on its ability to extract spatial hierarchies and classify complex handwritten patterns is examined.

1 Introduction

The present assignment explores an iterative machine learning workflow using Convolutional Neural Networks (CNNs) on the MNIST dataset. Starting with a robust baseline model and progressively increasing the network's depth, the improvements in performance, the reductions in loss, and the specific error patterns among the digits are analyzed.

2 Data, Preprocessing, and Workflow

The first and most decisive step in the development of any machine learning model is the correct preparation of the data.

- **Dataset:** The classic MNIST dataset (mnist_784), which includes 28×28 grayscale images of handwritten digits (0 to 9).
- **Preprocessing:** The original pixel values were normalized into the interval $[0, 1]$, the images were reshaped into $(28, 28, 1)$ and the datasets were partitioned (stratified splitting). This process is crucial for the smooth and rapid convergence of the optimization algorithm.
- **Workflow & Architecture:** During training, the adam optimization algorithm was configured (as it automatically adjusts the learning rate per parameter, ensuring faster and more stable convergence), the `sparse_categorical_crossentropy` loss function was used, and an `EarlyStopping` callback was employed to monitor convergence.
- **Evaluation:** The evaluation was carried out via the Test Accuracy, the Test Loss, the training/validation curves, and detailed confusion matrices.

3 Model 1: Baseline Architecture and Evaluation

3.1 Architecture and Depth

The baseline architecture (Model 1) was developed with a single, foundational convolutional block. It consists of a **Conv2D** layer for feature extraction, followed by a **MaxPooling2D** layer for spatial downsampling, a **Dropout (0.3)** layer for regularization (in order to prevent overfitting), a **Flatten** layer, and fully connected (**Dense**) classification layers ending in a softmax output.

3.2 Training

The evaluation of Model 1 on the Test Set yielded an Accuracy of 98.7% and a Loss of 0.046. Studying the training curves (Figure 1), smooth convergence and stable learning dynamics are observed throughout the epochs.

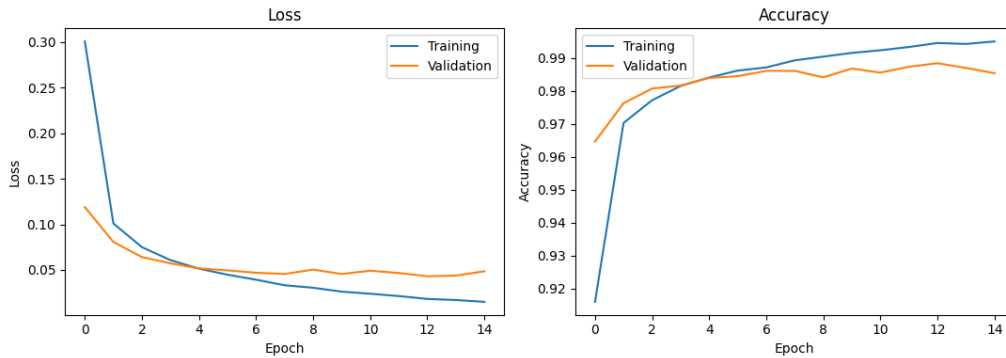


Figure 1: Training and Validation Loss and Accuracy curves for Model 1 (Baseline CNN).

3.3 Confusion Matrix and Error Analysis

Examining the elements outside the main diagonal of the Confusion Matrix, specific patterns are identified. Model 1 presented a total of 184 misclassifications, with digit 9 noting the highest number of confusions (13 errors). These errors are fully explained based on the morphology of the digits, as geometrically similar digits may be confused due to natural deviations in handwriting.

4 Model 2: Deeper Architecture and Evaluation

4.1 Architecture and Depth

In the second section, an attempt was made to address the weaknesses of Model 1, testing whether increasing the network's depth enhances feature representation. Model 2 introduces an additional convolutional layer (**Conv2D**) combined with supplementary feature mapping, before pooling and classification. This deeper topology allows the network to capture more complex spatial hierarchies.

4.2 Training

The performance of Model 2 was superior, achieving an Accuracy of 99.0% and a reduced Loss equal to 0.035, proving that the deeper architecture increased the prediction confidence.

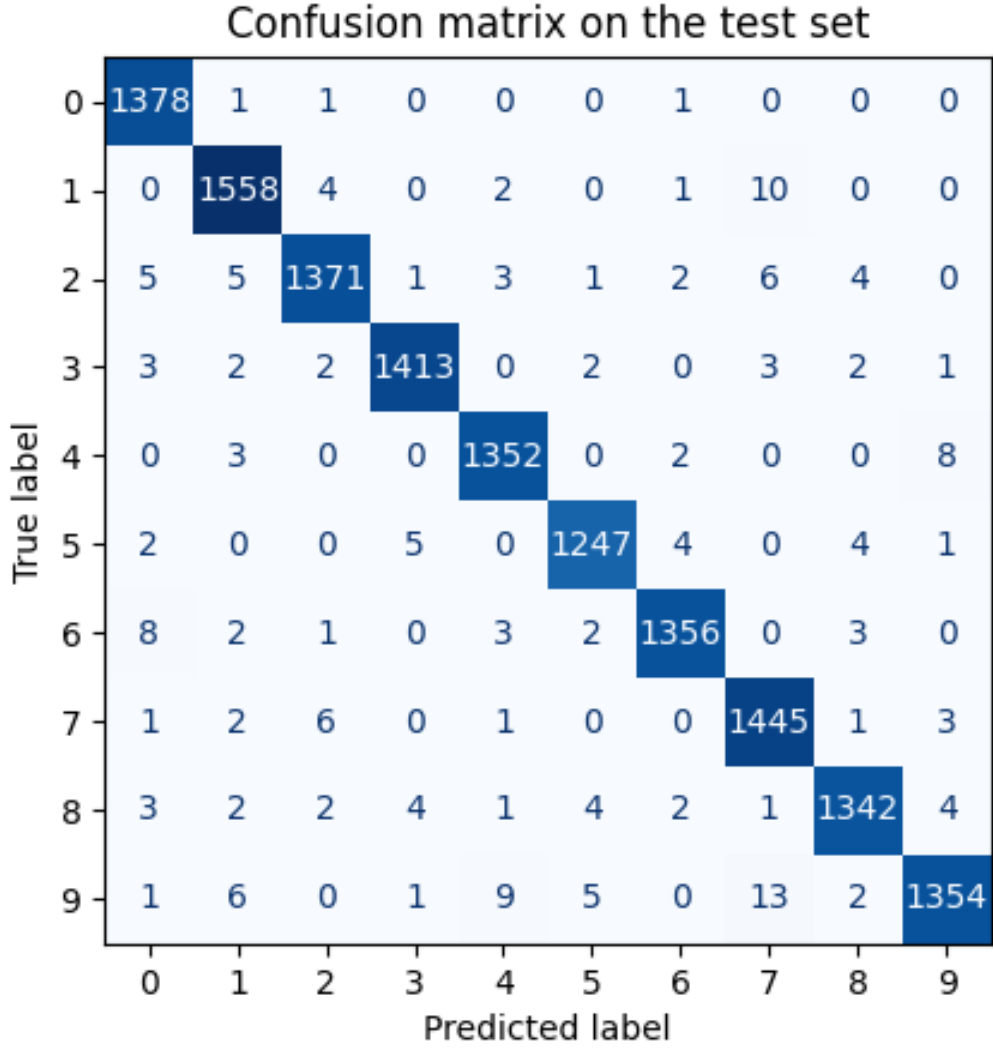


Figure 2: Confusion Matrix visualization for Model 1.

	0	1	2	3	4	5	6	7	8	9
Total Errors	3	17	27	15	13	16	19	14	23	37
Max Single Error	1	10	6	3	8	5	8	6	4	13
184										

Figure 3: Error summary per digit (Total Errors and Max Single Error) for Model 1 (184 total errors).

4.3 Confusion Matrix and Error Analysis

After the architectural upgrade, the misclassifications were re-evaluated. The total errors were reduced to 144, reflecting the improved separation between structurally similar digits.

5 Comparison and Final Conclusions

5.1 Comparative Summary

The final stage of the analysis concerns the direct comparison of the two models. The transition from Model 1 to Model 2 yielded clear quantitative improvements. Although the absolute

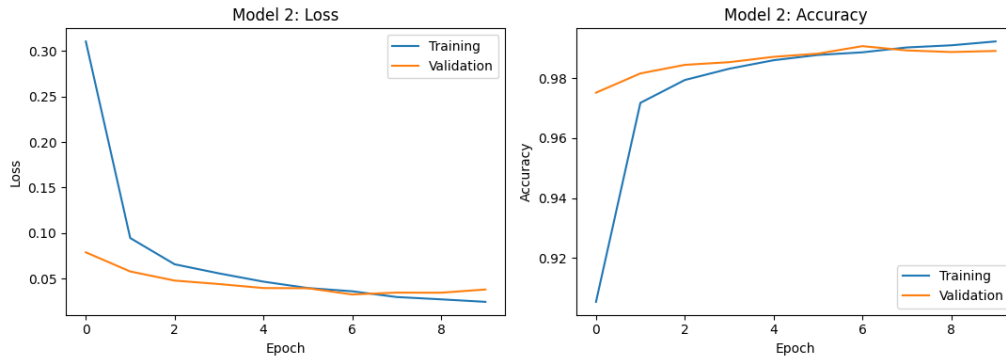


Figure 4: Training and Validation Loss and Accuracy curves for Model 2 (Deeper CNN).

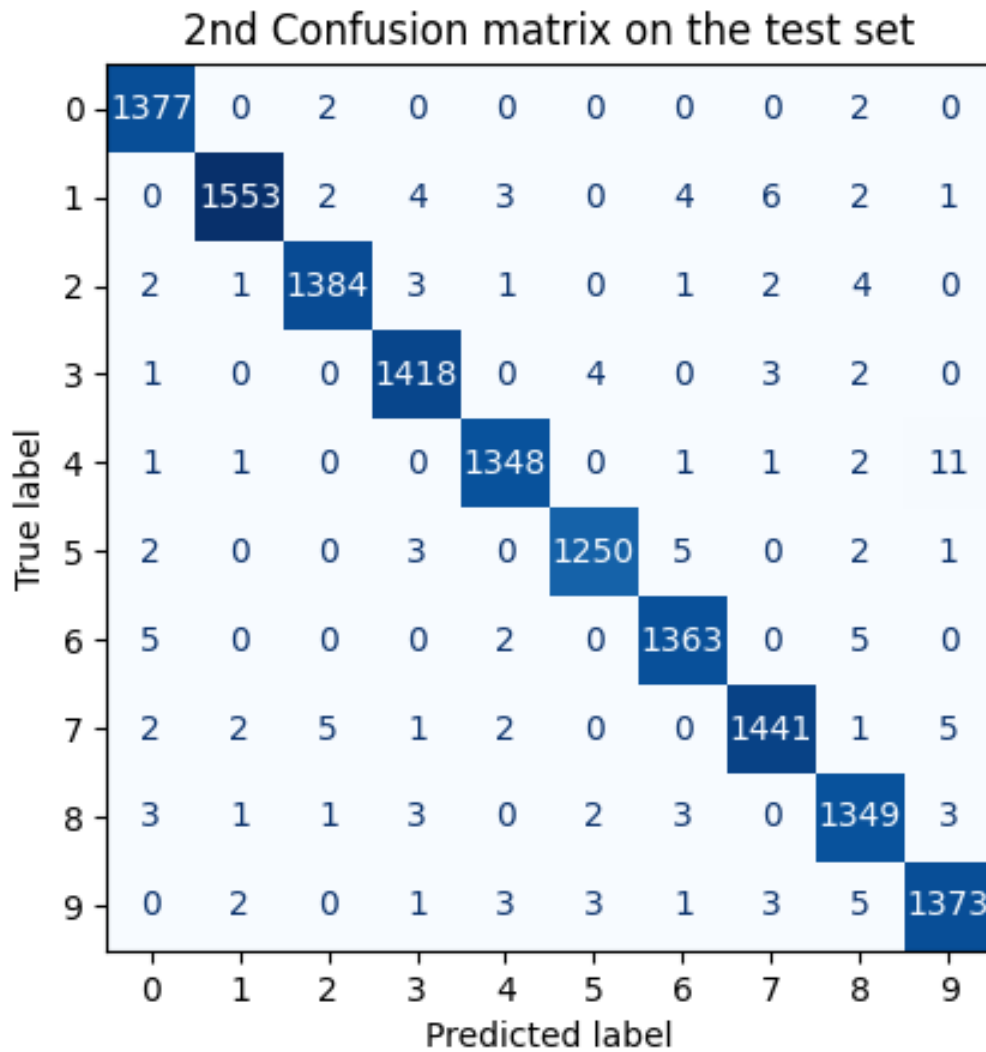


Figure 5: Confusion Matrix visualization for Model 2.

reduction by 40 errors (from 184 to 144) might seem modest, it translates to a significant **reduction of the error rate by 21.7%**, alongside the reduction of the Test Loss (from 0.046 to 0.035).

	0	1	2	3	4	5	6	7	8	9
Total Errors	4	22	14	10	17	13	12	18	16	18
Max Single Error	2	6	4	4	11	5	5	5	3	5

144

Figure 6: Error summary per digit (Total Errors and Max Single Error) for Model 2 (144 total errors).

5.2 Reflection

- **What worked well:** The architectural approach yielded excellent results, successfully combining the prevention of overfitting through the synergy of Dropout (0.3) and EarlyStopping. Specifically, the addition of just one extra convolutional layer (Conv2D) to the second model achieved a 21.7% reduction in total errors, without disproportionately increasing training time or causing instability in convergence.
- **Challenges:** During implementation, challenges arose regarding hyperparameter tuning. Furthermore, inherent separation difficulties were identified in geometrically similar digits (such as the confusion between 4 and 9), which are due to natural variations in human handwriting.
- **Future Improvements:** For the further evolution and optimization of the system, the application of data augmentation techniques is proposed to better address distortions. Additionally, for future deployment in a real-world operational context, it is deemed necessary to train the model with samples from different social groups and different cultural/geographic backgrounds (different writing styles, alphabets), in order to ensure maximum robustness and generalization capability.